The Transformer Architecture

George Michailidis

gmichail@ucla.edu

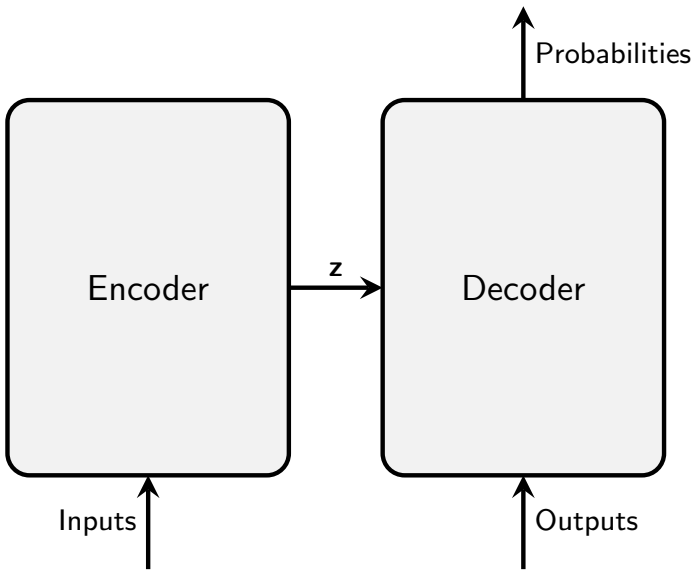
STAT 426

Motivation: Why Transformers?

- Prior State of the Art (RNNs/LSTMs):
  - Strictly sequential computation: $h_t = f(h_{t-1}, x_t)$.
  - Precludes parallelization within training examples (a bottleneck for modern GPUs).
  - Long-range dependencies are notoriously hard to learn due to information decay over time.
- The Transformer Solution (Vaswani et al., 2017):
  - Eschew recurrence entirely.
  - Rely solely on attention mechanisms to draw global dependencies, creating paths of length $O(1)$ between any two tokens in a sequence.
  - "Attention Is All You Need."

High-Level Architecture

- Encoder-Decoder Structure:
  - Encoder: Maps an input sequence $(x_1, \ldots, x_n)$ to a sequence of continuous representations $\mathbf{z} = (z_1, \ldots, z_n)$.
  - Decoder: Given $\mathbf{z}$, generates an output sequence $(y_1, \ldots, y_m)$ in an auto-regressive manner.
- Key Innovations:
  - Stacked Self-Attention
  - Point-wise Feed-Forward Networks
  - Positional Encoding
  - Residual Connections
  - Layer Normalization



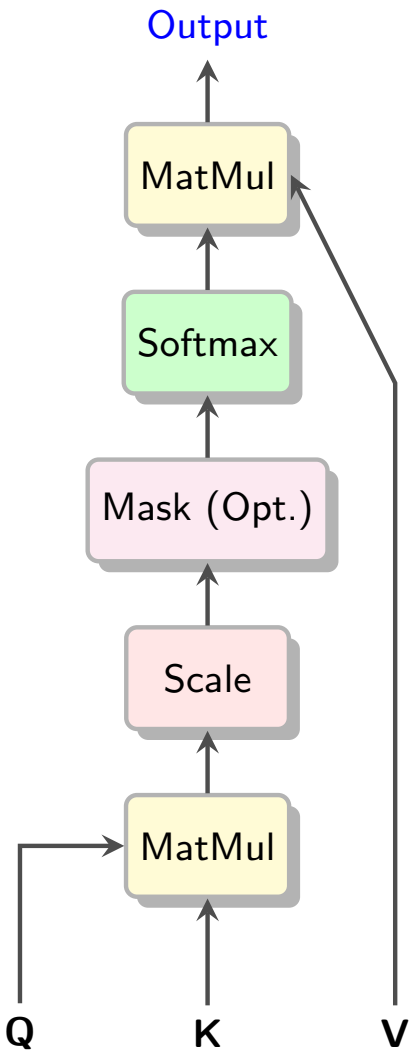
Recap: Scaled Dot-Product Attention

The Math:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

Key Components:

- Query ($\mathbf{Q}$): What we are actively looking for.
- Key ($\mathbf{K}$): What the data is labeled as (the identifiers).
- Value ($\mathbf{V}$): The actual data content.
- Scale ($\frac{1}{\sqrt{d_k}}$): Stabilizes the variance of the dot products to prevent Softmax saturation.



Attention Breakdown: The First "MatMul"

- The Operation: $\mathbf{QK}^\top$
- What it does: Calculates the dot product between every Query and every Key.
- Intuition: This is a similarity check. A high dot product means the Query closely aligns with a particular Key. It answers the question: *"How relevant is this specific piece of historical data to what I am looking for right now?"*
- Result: Yields a matrix of raw, unnormalized "attention scores" representing the alignment between all tokens.

Detailed Recap of the Key Equation

The Attention Formula:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

Dimensions: Time $\times$ Features

- $\mathbf{Q} \in \mathbb{R}^{T_q \times d_k}$: The **Queries** ($T_q$ = number of target tokens, d_k = feature dimension).
- $\mathbf{K} \in \mathbb{R}^{T_k \times d_k}$: The **Keys** ($T_k$ = number of source tokens).
- $\mathbf{V} \in \mathbb{R}^{T_k \times d_v}$: The **Values** (typically $d_v = d_k$, sharing the T_k sequence length).

Attention Breakdown: Scale and Mask

- Scale ($\div\sqrt{d_k}$):
  - As dimensions (d_k) grow, dot products naturally produce larger magnitude values.
  - Large values push the Softmax function into regions with extremely small gradients (vanishing gradients).
  - Dividing by the square root of the dimension stabilizes the variance and ensures healthy gradient flow.
- Mask (Optional):
  - Used primarily in the Decoder (or for padded tokens).
  - Replaces specific alignment scores with $-\infty$.
  - Ensures the model cannot "look into the future" during autoregressive generation, preserving the causal flow of time.

Attention Breakdown: Softmax and Final MatMul

- Softmax:
  - Converts the raw, scaled scores into a probability distribution.
  - Applied row-wise: For each Query (row), the attention weights distributed over all Keys (columns) sum exactly to 1.0.
  - Tokens with scores of $-\infty$ (from the Mask) become exactly 0.
- Final MatMul ($\times \mathbf{V}$):
  - We multiply the resulting probability weights by the Values ($\mathbf{V}$).
  - Intuition: We take a *weighted sum* of the actual data. If a specific token's Key had a 90% match with our Query, its Value will make up 90% of the final output vector for that position. Irrelevant tokens are effectively filtered out.

Recap: Multi-Head Attention (MHA)

- Concept: Instead of performing a single attention function, linearly project the queries, keys, and values h times with different, learned linear projections.
- Why? It allows the model to jointly attend to information from different representation subspaces at different positions. With a single attention head, averaging inhibits this.
- Analogy: Similar to having multiple feature kernels in a CNN, where each kernel looks for different structural patterns.
- Equation:

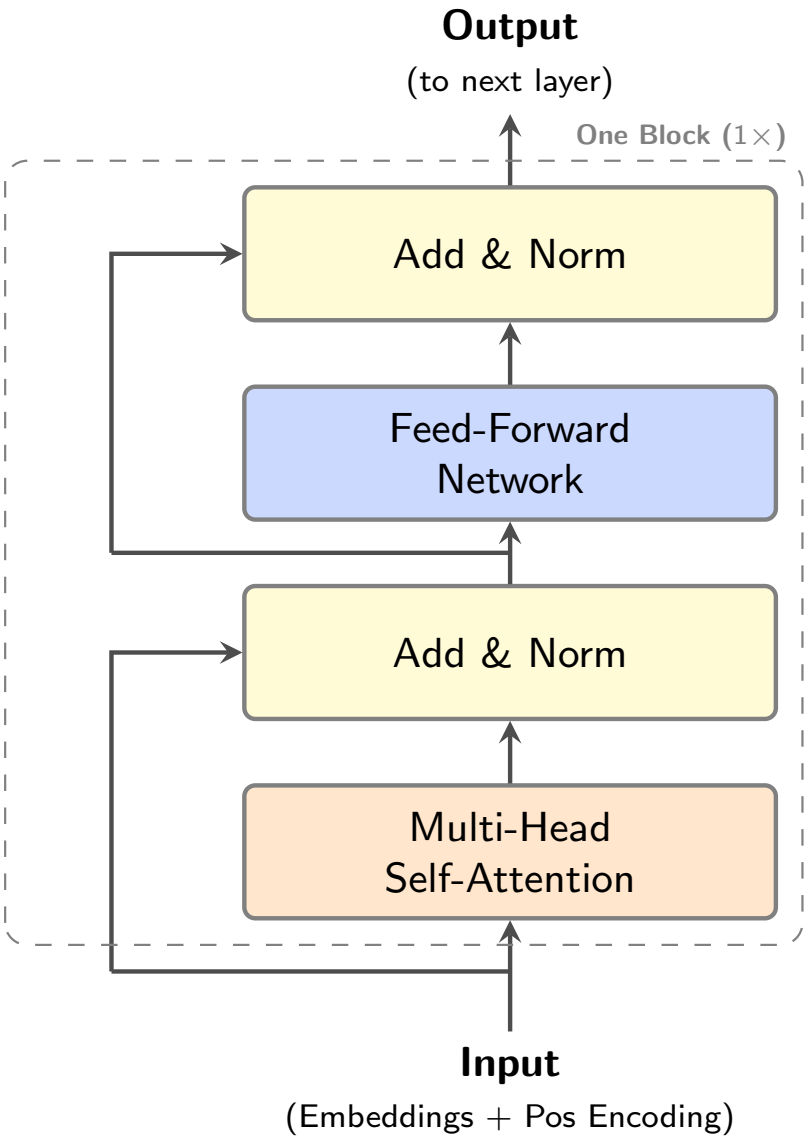
$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\mathbf{W}^O$$
$$\text{where head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V)$$

Zooming Out: The Micro, Meso and Macro View

Before we look at the entire stack, let's understand how the pieces fit together.

- 1. The Microscopic View (The Math):
  - The Scaled Dot-Product Attention ($\mathbf{QK}^\top / \dots$) we just covered.
  - This is the fundamental engine that calculates relevance between tokens.
- 2. The Meso View (The Sub-layers):
  - We bundle multiple attention engines together to create **Multi-Head Attention**.
  - We pair this with a **Feed-Forward Network** for token-level processing.
- 3. The Macro View (The Block & The Stack):
  - We wrap these sub-layers in structural "plumbing" (Add & Norm) to create a single **Encoder Block**.
  - Finally, we stack N of these blocks on top of each other to form the full Encoder.

Anatomy of a Single Encoder Block



Following the Data Flow:

1. Data Enters: The sequence tensor X enters the block. *(Note: If this is block 1, X already includes Positional Encoding!)*
2. Self-Attention: Tokens mix information. The output is Attention(X).
3. Add & Norm 1: The original input X travels via the **residual connection** (left arrow) and is added to the attention output, then normalized.
   $H_1 = \text{LayerNorm}(X + \text{Attention}(X))$
4. Feed-Forward: The normalized data H_1 is processed point-wise.
5. Add & Norm 2: H_1 bypasses the FFN via the second residual connection, is added to the FFN output, and normalized.
   $H_2 = \text{LayerNorm}(H_1 + \text{FFN}(H_1))$

Positional Encoding

- The "Bag of Words" Problem: Recall that Multi-Head Attention operates on all tokens simultaneously. Without explicit sequence data, the model would not know if a word is at the beginning or end of a sentence.
- The Injection Point: Positional Encoding happens before the sequence enters the first Encoder block.
- The Mechanism:
  - We take the sequence of token Embeddings (E).
  - We generate a geometric Positional Encoding matrix (PE) of the exact same dimensions using sine and cosine waves.
  - The Input to the first block is simply: $X = E + PE$.
- Subsequent Layers: For Blocks 2 through N, the position data does not need to be re-injected; it is preserved and passed upward via the residual connections.

The Add & Norm Block - I

Every sub-layer (Self-Attention or FFN) is wrapped in a residual connection and layer normalization.

The "Add" Residual Connection

- **The Problem:** Deep networks suffer from vanishing gradients as the chain rule multiplies small numbers during backpropagation.
- **The Solution:** Adding the input back to the output ($\mathbf{x} + \text{Sublayer}(\mathbf{x})$) creates a linear bypass. Gradients flow backwards completely unchanged, allowing us to stack dozens of layers.
- *Requirement:* To add $\mathbf{x}$ to $\text{Sublayer}(\mathbf{x})$, both must have the exact same dimension ($d_{\text{model}} = 512$).

Architectural Evolution: Post-LN vs. Pre-LN

- **Original (Post-LN):** $\text{Output} = \text{LayerNorm}(\mathbf{x} + \text{Sublayer}(\mathbf{x}))$
- **Modern (Pre-LN):** $\text{Output} = \mathbf{x} + \text{Sublayer}(\text{LayerNorm}(\mathbf{x}))$
- *Why the change?* Models like GPT and LLaMA use Pre-LN because it leaves the residual highway ($\mathbf{x} + \dots$) completely uninterrupted by normalization layers. This dramatically improves training stability for very deep networks.

The Add & Norm Block - II

Without normalization, values would grow unbounded as we continuously add residual connections layer after layer.

BatchNorm (Standard in CNNs)

- Computes stats across the *batch* dimension.
- **Flaw for NLP:** Problematic because sentence lengths vary widely (padding issues), and batch statistics are highly noisy for rare tokens.

LayerNorm (Standard in NLP)

- Computes stats across the *feature* dim (d_{model}) for a *single* token.
- **Advantage:** Independent of batch size and sequence length. Stabilizes hidden state dynamics globally.

Math Equation For a specific token vector $\mathbf{h} \in \mathbb{R}^{d_{\text{model}}}$:

$$\text{LN}(\mathbf{h}) = \gamma \frac{\mathbf{h} - \mu}{\sigma} + \beta$$

Where μ and σ are scalar stats of $\mathbf{h}$, and γ, β are learnable scaling and shifting vectors.

The Feed-Forward Network (FFN)

If Self-Attention is how tokens "talk" to each other, the FFN is how each token "processes" what it learned from the conversation.

- "Position-wise": The FFN is applied to each token's position i independently but identically (using the exact same shared weights $\mathbf{W}_1, \mathbf{W}_2$).

The Expansion-Compression Operation

$$\text{FFN}(\mathbf{x}) = \text{Activation}(\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2$$

- **Step 1 (Expand):** Project the d_{model}-dimensional token (e.g., 512) up to a much wider hidden dimension (e.g., $d_{\text{ff}} = 2048$).
- **Step 2 (Non-Linearity):** Apply an activation function (like ReLU or GELU). Self-Attention is entirely linear; this step is crucial for learning complex, non-linear representations.
- **Step 3 (Compress):** Project back down to $d_{\text{model}} = 512$ so the vector can be seamlessly added back into the residual stream.

Digression: Activation Functions (ReLU vs. GELU)

While the original Transformer used ReLU, modern architectures (BERT, LLaMA, GPT) overwhelmingly rely on GELU or its variants (like SwiGLU).

ReLU (Rectified Linear Unit)

- **Formula:** $f(x) = \max(0, x)$
- **Behavior:** Strict gating. If $x < 0$, the output is exactly 0 (a hard switch).
- **Pros:** Computationally extremely cheap.
- **Cons:** "Dead Neurons." If weights push a unit deeply negative, its gradient becomes zero and it stops learning entirely.

GELU (Gaussian Error Linear Unit)

- **Formula:** $f(x) = x \cdot \Phi(x)$
  ($\Phi(x)$ is the CDF of the standard normal distribution)
- **Behavior:** Probabilistic gating. Weights inputs by their value, modulated by the probability that the input is greater than other randomly drawn inputs.
- **Advantage:** Continuously differentiable (smooth) everywhere. Allows a small amount of negative info to flow, creating a smoother optimization landscape.

Example: Setup & Hyperparameters

Let's trace the dimensions strictly for a Time Series scenario.

- Scenario: We have 10 hours of data ($T = 10$) with 2 features (Temp, Pressure).
- Hyperparameters:
  - Sequence Length (T): 10
  - Raw Features (F_{raw}): 2
  - Model Dimension (d_{model}): 6 (Artificially small for demonstration)
  - Heads (h): 3
  - Head Dimension (d_k): $d_{model}/h = 2$

Step 1: Raw Input (X_{raw})

$$X_{raw} \in \mathbb{R}^{10\times 2} = \begin{bmatrix} \text{temp}_1 & \text{pres}_1 \\ \vdots & \vdots \\ \text{temp}_{10} & \text{pres}_{10} \end{bmatrix}$$

Why Positional Encoding?

- The Transformer architecture contains no recurrence and no convolution.
- Without positional information, the model is entirely permutation invariant.
- The Problem: "Dog bites Man" would look mathematically identical to "Man bites Dog" to the self-attention mechanism (acting simply as a parallel bag-of-words).
- The Solution: We must explicitly inject structural information about the relative or absolute position of tokens. We do this by adding a "positional vector" to the input embeddings at the very bottom of the stack.

Step 1: Sinusoidal Positional Encoding

- Key idea: Create a unique geometric "barcode" for every position in the sequence using sine and cosine waves of varying frequencies.
- Math Formula:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$
$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

- pos: Position of the token in the sequence $(0, 1, 2, \dots)$.
- i: Dimension index $(0 \leq 2i < d_{model})$.

Properties of Sinusoidal Encoding

- Wavelengths: Form a continuous geometric progression from 2π to $10000 \cdot 2\pi$. The early dimensions oscillate quickly; the later dimensions oscillate incredibly slowly.
- Relative Positions (The Magic Property): For any fixed offset k, PE_{pos+k} can be represented exactly as a linear transformation of PE_{pos}.

$$\begin{bmatrix} \sin(pos+k) \\ \cos(pos+k) \end{bmatrix} = \begin{bmatrix} \cos k & \sin k \\ -\sin k & \cos k \end{bmatrix} \begin{bmatrix} \sin(pos) \\ \cos(pos) \end{bmatrix}$$

- Why this matters: This specific rotation matrix property ensures that the model can easily learn to attend to tokens by *relative* distances (e.g., "three tokens ago") rather than relying purely on absolute positions.

Evolution of Positional Encodings: Context Extrapolation - I

The original Sinusoidal encodings theoretically support infinite sequences, but in practice, models struggle to *extrapolate* to lengths beyond their training data.

Modern architectures have largely moved to the following paradigms:

Evolution of Positional Encodings: Context Extrapolation - II

1. Learned Absolute Embeddings (BERT, GPT-2/3)
   - Instead of sine waves, the model simply learns a unique, trainable vector for $pos = 1, pos = 2, \ldots, pos = N$.
   - **Extrapolation:** None. If trained on 2048 tokens, it physically cannot process token 2049 because no embedding exists for that position.
2. RoPE: Rotary Positional Embedding (LLaMA, Mistral)
   - Injects positional info directly into the Queries (Q) and Keys (K) at each layer by applying a mathematical rotation in the complex plane.
   - The dot product $Q_m \cdot K_n$ becomes a function only of the relative distance $(m - n)$.
   - **Extrapolation:** Good. Can be extended to massive contexts via "RoPE scaling" (interpolating the rotation angles) with minimal fine-tuning.
3. ALiBi: Attention with Linear Biases (BLOOM)
   - Discards input positional embeddings entirely. Instead, it subtracts a static, distance-based penalty directly from the pre-Softmax attention scores: $\text{score}(q_i, k_j) - m \cdot |i - j|$.
   - **Extrapolation:** Excellent. A model trained on 1K tokens can zero-shot evaluate on 10K+ tokens gracefully because it inherently penalizes distant tokens.

Example: Projection to Embeddings

Step 2: Creating the Sequence of Embeddings

The Transformer can not use raw features directly. We project them into the latent space $d_{model} = 6$ and add our Positional Encoding.

$$X = X_{raw}\mathbf{W}_e + \text{PosEncoding}$$

- $\mathbf{W}_e \in \mathbb{R}^{2\times 6}$ is a learnable projection matrix.
- Resulting Tensor $X \in \mathbb{R}^{10\times 6}$.

$$X = \begin{bmatrix} x_{1,1}, x_{1,2} & x_{1,3}, x_{1,4} & x_{1,5}, x_{1,6} \\ \vdots & \vdots & \vdots \\ x_{10,1}, x_{10,2} & x_{10,3}, x_{10,4} & x_{10,5}, x_{10,6} \end{bmatrix}$$

(Colors indicate which dimensions conceptually map to which head in the next step, though they are not separated here).

Example: The Multi-Head Split

Step 3: Splitting into 3 Heads

We project X into Queries/Keys/Values for each head. Effectively, the model splits the 6-dim vector into three 2-dim operational vectors.

Head 1	**Head 2**	**Head 3**
Focus: Dims 1-2	Focus: Dims 3-4	Focus: Dims 5-6
$Z_1 \in \mathbb{R}^{10\times 2}$	$Z_2 \in \mathbb{R}^{10\times 2}$	$Z_3 \in \mathbb{R}^{10\times 2}$

Each head runs Attention ($O(T^2)$) completely independently. Head 1 might learn to attend to "Volatility", Head 2 might track "Macro Trend". Let's zoom in on exactly how one head does this.

Remarks on Computing and Memory Complexity

While Multi-Head Attention is highly parallelizable, its standard dense implementation scales poorly with the sequence length (T).

1. Time Complexity: $\mathcal{O}(T^2 \cdot d)$
   - To compute the attention scores, we multiply the Query matrix $\mathbf{Q} \in \mathbb{R}^{T \times d}$ by the Key matrix $\mathbf{K}^\top \in \mathbb{R}^{d \times T}$ (where T is the number of tokens, and d is the embedding dimension).
   - Computing this $T \times T$ similarity matrix requires T^2 dot products, each of dimension d.
2. Memory Complexity: $\mathcal{O}(T^2)$
   - The resulting $T \times T$ attention weight matrix must be stored in memory to compute the Softmax and gradients.
   - The Scaling Problem: Doubling the context window *quadruples* the memory and compute requirements.
   - *Example:* Scaling from a $T = 2,000$ to $T = 100,000$ token sequence increases the attention matrix footprint by 2,500$\times$ per layer, per head.

Solution to the Computational and Memory Complexity Issue

1. Hardware Optimization: FlashAttention
   - Standard implementations physically write the massive $T \times T$ matrix to slow GPU High-Bandwidth Memory (HBM).
   - **FlashAttention** (Dao et al., 2022) uses hardware-aware "tiling" to compute the exact same mathematical result entirely in fast on-chip SRAM, bypassing the HBM read/write bottleneck.
2. Architectural Alternatives: Mamba & SSMs
   - Rather than optimizing the $\mathcal{O}(T^2)$ attention math, **Mamba** (Gu & Dao, 2023) replaces it entirely with Selective State Space Models (SSMs).
   - It compresses history into a hidden state, achieving **true linear** $\mathcal{O}(T)$ **scaling** in training and $\mathcal{O}(1)$ memory during generation, offering a strong alternative to Transformers for ultra-long sequences.

Step 4a: Projecting to Q, K, V (Head 1)

Zooming in on Head 1. Our sequence tensor is currently $X \in \mathbb{R}^{10\times 6}$.
To perform attention, Head 1 needs to create three distinct representations of the data: **Queries** (what I'm looking for), **Keys** (what I contain), and **Values** (what I actually am).

Head 1 uses three learnable weight matrices, each of shape (6×2): $\mathbf{W}_1^Q, \mathbf{W}_1^K, \mathbf{W}_1^V$.

The Matrix Multiplications

$$
\begin{aligned}
Q_1 = X \cdot \mathbf{W}_1^Q \quad &\rightarrow \quad \text{Dim: } (10 \times 6) \times (6 \times 2) = \mathbf{10} \times \mathbf{2} \\
K_1 = X \cdot \mathbf{W}_1^K \quad &\rightarrow \quad \text{Dim: } (10 \times 6) \times (6 \times 2) = \mathbf{10} \times \mathbf{2} \\
V_1 = X \cdot \mathbf{W}_1^V \quad &\rightarrow \quad \text{Dim: } (10 \times 6) \times (6 \times 2) = \mathbf{10} \times \mathbf{2}
\end{aligned}
$$

We now have 10 Queries, 10 Keys, and 10 Values. Each token has been compressed from a 6-dimensional embedding down to a 2-dimensional vector specifically for Head 1.

Step 4b: The Attention Scores

Now, Head 1 must calculate how much every timestep should "pay attention" to every other timestep.

We do this by taking the dot product of every Query against every Key. Mathematically, this is Q_1 multiplied by the transpose of K_1.

Computing the Raw Scores

$$\text{Scores}_1 = Q_1 \cdot K_1^T$$

$$\text{Dim: } (10 \times 2) \times (2 \times 10) = \mathbf{10} \times \mathbf{10}$$

- **Why** 10×10**?** We have 10 timesteps. The resulting matrix compares every timestep to every other timestep.
- Entry (i, j) represents how strongly the Query of token i aligns with the Key of token j.

Step 4c: Scaling and Softmax

The raw scores can get massive, which pushes the Softmax function into flat regions (killing gradients). We must scale them down.

1. Scale: We divide the entire 10×10 matrix by the square root of the head dimension ($\sqrt{d_k} = \sqrt{2}$).
2. Softmax: We apply the Softmax function across each *row*.

The Attention Weights (A_1)

$$A_1 = \text{softmax}\left(\frac{\text{Scores}_1}{\sqrt{2}}\right) \quad \rightarrow \quad \text{Dim: } \mathbf{10} \times \mathbf{10}$$

Every row now sums perfectly to 1.0. If row 1 is $[0.1, 0.8, 0.1, \ldots]$, it means Token 1 is paying 80% of its attention to Token 2!

Step 4d: Applying the Attention to the Values

We know the weights. Now we mix the actual data (the Values).

We multiply our 10×10 attention weight matrix by our 10×2 Values matrix.

The Output of Head 1 (Z_1)

$$Z_1 = A_1 \cdot V_1$$

$$\text{Dim: } (10 \times 10) \times (10 \times 2) = \mathbf{10} \times \mathbf{2}$$

For Token 1, its new 2-dimensional vector is a weighted average of *all* 10 Value vectors, using the percentages we calculated in the Softmax step.

Step 5: Recombining the Heads

While we meticulously tracked Head 1, Heads 2 and 3 were doing the exact same math in parallel!

- Head 1 produced $Z_1 \in \mathbb{R}^{10\times 2}$
- Head 2 produced $Z_2 \in \mathbb{R}^{10\times 2}$
- Head 3 produced $Z_3 \in \mathbb{R}^{10\times 2}$

Concatenation: We literally paste these matrices together side-by-side to rebuild our original model dimension ($d_{model} = 6$).

$$Z_{concat} = [Z_1, Z_2, Z_3] \quad \rightarrow \quad \text{Dim: } \mathbf{10} \times \mathbf{6}$$

Final Linear Mix: The heads operated in silos. To let them share what they learned, we multiply by an output weight matrix $\mathbf{W}^O \in \mathbb{R}^{6\times 6}$.

$$Z_{final} = Z_{concat} \cdot \mathbf{W}^O \quad \rightarrow \quad \text{Dim: } (10 \times 6) \times (6 \times 6) = \mathbf{10} \times \mathbf{6}$$

Step 6: The First "Add & Norm"

- The "Add" (Residual Connection): We take the original sequence tensor X (before it entered the attention mechanism) and add it element-wise to Z_{final}.
  - Both are strictly 10×6, so no matrix multiplication here—just simple addition!
  - $\text{Residual}_1 = X + Z_{final} \quad \rightarrow \quad$ Dim: $\mathbf{10} \times \mathbf{6}$
- The "Norm" (Layer Normalization): We normalize the values. Crucially, this happens across the 6 features for each of the 10 timesteps independently.
  - $H_1 = \text{LayerNorm}(\text{Residual}_1) \quad \rightarrow \quad$ Dim: $\mathbf{10} \times \mathbf{6}$

Checkpoint: Halfway through the Encoder block, our tensor H_1 is safely maintained at $\mathbf{10} \times \mathbf{6}$.

Step 7: The Feed-Forward Network (FFN)

Attention allowed tokens to talk to each other. The FFN allows each token to process what it learned individually.
Standard Transformer architecture sets the FFN hidden dimension (d_{ff}) to $4 \times d_{model}$. Here, $d_{ff} = 24$.

1. Expand (Linear 1): Multiply by $\mathbf{W}_1 \in \mathbb{R}^{6\times 24}$.

$$(10 \times 6) \times (6 \times 24) \rightarrow \mathbf{10} \times \mathbf{24}$$

2. Activate: Apply the non-linear ReLU or GELU function element-wise.

$$\text{Dim remains } \mathbf{10} \times \mathbf{24}$$

3. Compress (Linear 2): Multiply by $\mathbf{W}_2 \in \mathbb{R}^{24\times 6}$.

$$F = (10 \times 24) \times (24 \times 6) \rightarrow \mathbf{10} \times \mathbf{6}$$

Step 8: The Second "Add & Norm" (Block Complete!)

The final stabilizing wrapper.

- The "Add": We take the input to the FFN (H_1) and add it to the FFN output (F).

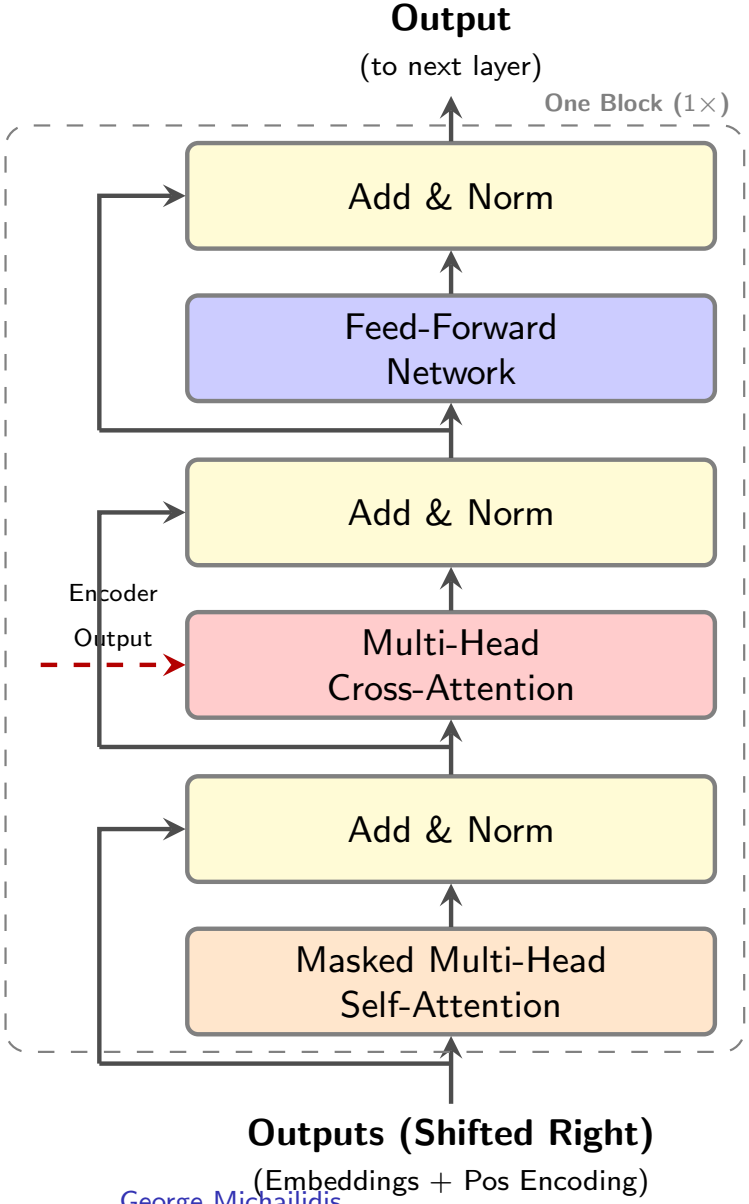
$$\text{Residual}_2 = H_1 + F \quad \rightarrow \quad \text{Dim: } \mathbf{10} \times \mathbf{6}$$

- The "Norm": Apply Layer Normalization one last time.

$$H_2 = \text{LayerNorm}(\text{Residual}_2) \quad \rightarrow \quad \text{Dim: } \mathbf{10} \times \mathbf{6}$$

The final output tensor H_2 is exactly $\mathbf{10} \times \mathbf{6}$. Because it is mathematically identical in shape to the original input X, we can stack dozens of these blocks on top of each other!

Anatomy of a Single Decoder Block



What makes the Decoder different?

1. Masked Self-Attention: The sequence Y attends to itself, but we strictly block tokens from "looking ahead" at future tokens.
2. Add & Norm 1: Standard residual bypass and layer normalization. $H_{d1} = \text{Norm}(Y + \text{MaskedAttn}(Y))$
3. **Cross-Attention: The Bridge.** The Decoder acts as the **Query** (asking for context), while the Encoder's final output provides the **Keys** and **Values**.
4. Add & Norm 2: $H_{d2} = \text{Norm}(H_{d1} + \text{CrossAttn}(H_{d1}, H_{enc}))$
5. Feed-Forward & Norm 3: Identical in structure to the Encoder's FFN, processing each token independently.

1. The Base: Inputs and Positional Encoding

Outputs (Shifted Right)

(Embeddings + Pos Encoding)

- "Outputs (Shifted Right)": The Decoder is auto-regressive; it predicts the next token based on previous tokens. During training, we feed it the target sequence, but shifted right by one position (starting with a <SOS> token) so it learns to predict token t using tokens 0 to $t-1$.
- Embeddings: Just like the Encoder, we map raw tokens into continuous vectors of dimension d_{model}.
- Positional Encoding: We add the exact same sinusoidal positional "barcodes" so the model knows the order of the generated sequence.

2. Masked Multi-Head Self-Attention

- The sequence attends to itself. Tokens look at other tokens in the generated sequence to gather context.
- **The Mask:** This is the crucial difference from the Encoder. When predicting the 3rd word, the model is strictly forbidden from looking at the 4th word.
- We enforce this by applying a look-ahead mask (setting future attention scores to $-\infty$) before the Softmax step, ensuring illegal "future" information is zeroed out.

Speeding up Training

The Decoder is strictly auto-regressive. It consumes its own previous outputs to generate the next one.

Looking at a language example:

- **Step 1:** Feed the model <SOS> (Start of Sequence). It predicts: `"The"`
- **Step 2:** Feed the model <SOS>, `"The"`. It predicts: `"cat"`
- **Step 3:** Feed the model <SOS>, `"The"`, `"cat"`. It predicts: `"sleeps"`

Key issue: If we train the model this way, it is incredibly slow (we lose all parallel computing power). Furthermore, if the model guesses wrong early on, the errors compound, making it impossible to learn later words effectively.

Teacher Forcing

As with other neural network architectures, during training, we do not let the model use its own bad guesses. We force it to use the correct answers.

Further, instead of waiting for the model to generate the sequence token-by-token, we feed the entire true target sequence into the Decoder all at once.
m

Why "Shifted Right"? To teach the model to predict token t, the input must

consist of tokens 0 through $t - 1$. Therefore, the input array is simply the true target array shifted to the right by one position, with a <SOS> token prepended to the start.

Visualizing the Shift

Assume our true target translation is: **"Le", "chat", "dort", <EOS>**

Time Step (t)	$t = 1$	$t = 2$	$t = 3$	$t = 4$
Decoder Input (Y_{raw})	<SOS>	Le	chat	dort
Expected Output (Target)	Le	chat	dort	<EOS>

Reading the Columns (What the model learns):

- At $t = 1$: Given <SOS>, you should predict **"Le"**.
- At $t = 2$: Given (<SOS>, "Le"), you should predict **"chat"**.
- At $t = 3$: Given (<SOS>, "Le", "chat"), you should predict **"dort"**.

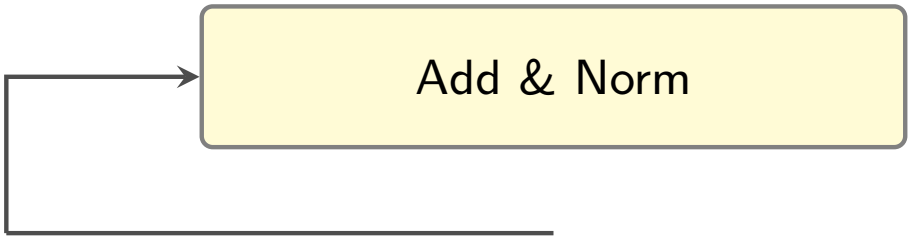
Tying it back to the Mask

However, if we feed the entire array [<SOS>, "Le", "chat", "dort"] into the Decoder all at once, the model would just "cheat" and look at the answer.

Need a Look-Ahead Mask

- Because Transformers process all tokens simultaneously, Token 1 (<SOS>) could mathematically peek at Token 2 ("Le") during the self-attention phase to figure out what to predict.
- The Mask (the $-\infty$ upper triangle we discussed) strictly blocks this.
- The End Result: We get the massive speed benefits of parallel matrix multiplication (processing all 4 timesteps simultaneously), while mathematically guaranteeing the model only uses past context to predict the future.

3. The First Add & Norm

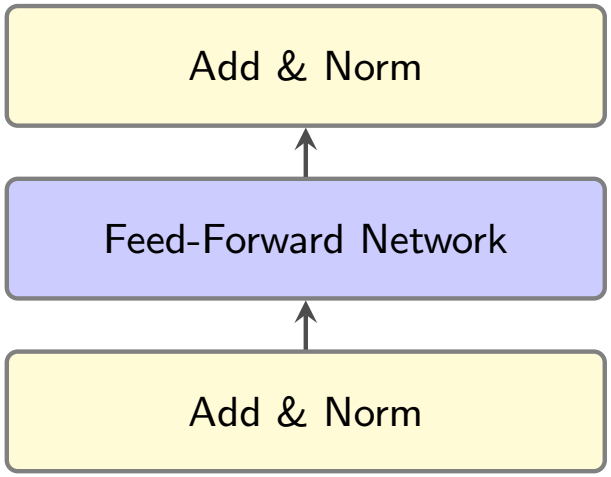


- The Residual Connection ("Add"): The original embeddings bypass the masked attention block and are added directly to the attention output. This prevents vanishing gradients in deep networks.
- Layer Normalization ("Norm"): We normalize the mean and variance of the features for each token independently. This stabilizes the training process.
- We now have a safe, causally-masked contextual representation of what the Decoder has generated so far.

4. Multi-Head Cross-Attention

- The Bridge: This is where the Encoder and Decoder finally meet.
- Queries (Q): Come from the Decoder's previous layer. (e.g., "I am currently translating a verb, what noun does it apply to?")
- Keys (K) & Values (V): Come straight from the top of the Encoder. (e.g., "Here is the rich contextual memory of the entire original input sentence.")
- The Decoder dynamically searches the Encoder's memory to figure out what to generate next.

5. Feed-Forward Network & Final Stabilization



- Add & Norm 2: We stabilize the output of the Cross-Attention block with another residual connection and layer norm.
- Feed-Forward Network: Identical to the Encoder. A two-layer neural network with a ReLU/GELU activation applies a non-linear transformation to each token's vector independently.
- Add & Norm 3: One final stabilization. The resulting tensor is now ready to be passed up to the next Decoder block, or to the final output layer!

Example: Setup & Hyperparameters

Now that we know the blocks, let's track the matrix math. Assume we are predicting a target sequence of length 5, given our previous 10-step Encoder history.

- Encoder Output (H_{enc}): Our final memory tensor from the Encoder. Dim: $\mathbf{10} \times \mathbf{6}$.
- Target Sequence Length (M): 5
- Decoder Input (Y_{raw}): The target sequence, shifted right.

Step 1: Embeddings & Positional Encoding

We project the target sequence into our $d_{model} = 6$ latent space and add our Positional Encoding.

$$Y = Y_{raw}\mathbf{W}_e + \text{PosEncoding} \quad \rightarrow \quad \text{Dim: } \mathbf{5} \times \mathbf{6}$$

Current State: The Decoder is starting at the bottom with a tensor $Y \in \mathbb{R}^{5\times 6}$.

Step 2: Masked Attention Math (Zooming on Head 1)

Head 1 projects Y into Q_d, K_d, V_d using its own learnable weights.

$$
\begin{aligned}
Q_d &= Y\mathbf{W}_{d1}^{Q} \quad \rightarrow \quad (5 \times 6) \times (6 \times 2) = \mathbf{5} \times \mathbf{2} \\
K_d &= Y\mathbf{W}_{d1}^{K} \quad \rightarrow \quad (5 \times 6) \times (6 \times 2) = \mathbf{5} \times \mathbf{2} \\
V_d &= Y\mathbf{W}_{d1}^{V} \quad \rightarrow \quad (5 \times 6) \times (6 \times 2) = \mathbf{5} \times \mathbf{2}
\end{aligned}
$$

Computing Raw Scores

$$
\text{Scores}_d = Q_d K_d^T \quad \rightarrow \quad (5 \times 2) \times (2 \times 5) = \mathbf{5} \times \mathbf{5}
$$

This 5×5 matrix shows how the 5 target tokens relate to each other. But the upper triangle contains "future" information we must hide!

Step 3: Applying the Look-Ahead Mask

We force the model to ignore the future by adding a mask matrix consisting of zeros and negative infinities.

$$\text{Mask} = \begin{bmatrix} 0 & -\infty & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty & -\infty \\ 0 & 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

When we compute the Softmax:

$$A_d = \text{softmax}\left(\frac{\text{Scores}_d}{\sqrt{2}} + \text{Mask}\right) \quad \rightarrow \quad \mathbf{5} \times \mathbf{5}$$

The Softmax of $-\infty$ is strictly 0. This guarantees that Token 2 places exactly 0% of its attention weight on Tokens 3, 4, and 5!

Step 4: Output of Masked Attention & First Norm

We finish the self-attention exactly as we did in the Encoder.

- Multiply attention weights by Values:

$$Z_{d1} = A_d V_d \quad \rightarrow \quad (5 \times 5) \times (5 \times 2) = \mathbf{5} \times \mathbf{2}$$

- Concatenate Heads & Linear Mix: Recombine all 3 heads to get $Z_{self} \in \mathbb{R}^{5 \times 6}$.
- Add & Norm 1:

$$H_{d1} = \text{LayerNorm}(Y + Z_{self}) \quad \rightarrow \quad \mathbf{5} \times \mathbf{6}$$

Checkpoint: Our tensor H_{d1} is 5×6. It contains safe, causally-masked contextual information about the target sequence so far.

Step 5: Cross-Attention Math (Tracking the Dimensions)

This is where the $T = 10$ and $M = 5$ sequences interact.

Head 1 projects the components into its $d_k = 2$ subspace:

$$Q_{cross} = H_{d1}\mathbf{W}_c^Q \quad \rightarrow \quad (5 \times 6) \times (6 \times 2) = \mathbf{5} \times \mathbf{2}$$
$$K_{cross} = H_{enc}\mathbf{W}_c^K \quad \rightarrow \quad (\mathbf{10} \times 6) \times (6 \times 2) = \mathbf{10} \times \mathbf{2}$$
$$V_{cross} = H_{enc}\mathbf{W}_c^V \quad \rightarrow \quad (\mathbf{10} \times 6) \times (6 \times 2) = \mathbf{10} \times \mathbf{2}$$

The Cross-Attention Scores

$$\text{Scores}_c = Q_{cross}K_{cross}^T \quad \rightarrow \quad (5 \times 2) \times (2 \times 10) = \mathbf{5} \times \mathbf{10}$$

A 5×10 matrix! For every single one of the 5 Decoder tokens, we have generated an attention score for all 10 Encoder tokens.

Step 6: Finalizing Cross Attention & Second Norm

Now we extract the information from the Encoder's Values.

- Scale & Softmax: Apply softmax to the 5×10 matrix so each row sums to 1.

$$A_c = \text{softmax}\left(\frac{\text{Scores}_c}{\sqrt{2}}\right) \quad \rightarrow \quad \mathbf{5 \times 10}$$

- Apply to Values:

$$Z_{c1} = A_c V_{cross} \quad \rightarrow \quad (5 \times 10) \times (10 \times 2) = \mathbf{5 \times 2}$$

We concatenate the heads (5×6), apply the final linear mix, and do our second residual Add & Norm to get $H_{d2} \in \mathbb{R}^{5 \times 6}$.

Step 7: Feed-Forward, Final Norm, and Generation

Just like the Encoder, we process each token's 6-dimensional vector independently through a hidden layer of size 24.

1. Expand, Activate, Compress:

$$H_{out} = \text{FFN}(H_{d2}) \quad \rightarrow \quad \mathbf{5 \times 6}$$

2. Add & Norm 3: Final residual connection and layer norm yields our completed Decoder block output: $H_{out} \in \mathbb{R}^{5\times 6}$.

Linear Projection & Softmax (The End) Suppose we have a vocabulary of size $V_{vocab} = 1000$. We multiply H_{out} by $\mathbf{W}_{logits} \in \mathbb{R}^{6\times 1000}$.

$$\text{Logits} = H_{out}\mathbf{W}_{logits} \quad \rightarrow \quad \mathbf{5 \times 1000}$$

Apply Softmax to get a probability distribution over the 1000 possible next tokens for each of our 5 timesteps!

Training Step 1: The Forward Pass

Now let us zoom out and look at the entire training loop for a single mini-batch.

- **Given:** A pair of sequences. *Source:* "The cat sleeps" $\rightarrow$ *Target:* "Le chat dort <EOS>"
- **Encoder:** Processes "The cat sleeps" all at once to produce H_{enc}.
- **Decoder:** Receives the shifted target [<SOS>, "Le", "chat", "dort"] all at once.
- **Outputs:** The Decoder outputs a massive tensor of Logits.

$$\text{Logits} \in \mathbb{R}^{M \times V_{vocab}} \quad \rightarrow \quad \mathbf{4 \times 1000}$$

The model has simultaneously generated 4 probability distributions, one for each timestep.

The Final Step: Where does the "1000" come from?

The model's internal math uses a latent dimension of $d_{model} = 6$. But we need it to output actual words in a language task.

- Vocabulary Size (V_{vocab}): Before training, we define a master dictionary of every token (word or sub-word) the model is allowed to know. In our example, we assumed our dictionary has 1000 total words.
- The Final Linear Layer: To translate from the 6-dimensional latent space back into human vocabulary, we multiply the Decoder's final output by a massive projection matrix, $\mathbf{W}_{logits} \in \mathbb{R}^{6 \times 1000}$.

The Dimension Shift

$$\text{Logits} = H_{out}\mathbf{W}_{logits} \quad \rightarrow \quad (\mathbf{4} \times 6) \times (6 \times \mathbf{1000}) = \mathbf{4} \times \mathbf{1000}$$

What this matrix actually means: For each of the 4 time steps, the model has generated an array of 1,000 raw scores. We pass these through a **Softmax** function to convert them into 1,000 percentages. The dictionary word with the highest percentage becomes the model's prediction!

The Final Step: Adapting to Time Series Data (Regression)

The model's internal math uses a latent dimension of $d_{model} = 6$. But instead of predicting words, we need to output continuous numerical values (e.g., future temperatures in our toy example).

- Target Variables (D_{out}): In NLP, we had a vocabulary dictionary. In time series, we simply define how many continuous variables we are predicting at each step. If we are just predicting temperature, $D_{out} = 1$.
- The Final Linear Layer: To translate from the 6-dimensional latent space into a real-world numerical forecast, we multiply the Decoder's final output by a much smaller projection matrix, $\mathbf{W}_{out} \in \mathbb{R}^{6\times 1}$.

The Dimension Shift

$$\text{Predictions} = H_{out}\mathbf{W}_{out} \quad \rightarrow \quad (\mathbf{4} \times 6) \times (6 \times \mathbf{1}) = \mathbf{4} \times \mathbf{1}$$

What this matrix actually means: For each of the 4 future timesteps, the model has generated exactly one raw continuous number (e.g., 72.5, 73.1, 71.8, 70.2). Crucial Difference: We do not use a Softmax function! Softmax is for probabilities. Instead, these raw numbers are our actual predictions. We evaluate them using Mean Squared Error (MSE) rather than Cross-Entropy Loss (see next slide).

Training Step 2: Cross-Entropy Loss

How do we penalize the model for wrong guesses?

For each of the 4 time steps, we compare the model's predicted probability distribution to the actual true word.

Calculating the Loss At $t = 1$, the true target is "Le" (let's say it's index 42 in our vocab).

- The model's Softmax output gave "Le" a probability of 0.15.
- **Loss** $= -\log(0.15) = 1.89$

We calculate this loss for all 4 timesteps simultaneously and average them. The goal of the optimizer is to push the probability of the correct indices as close to 1.0 as possible.

Training Step 3: Masking the Padding

Real-world training processes sequences in batches, but sentences have different lengths!

- To process a batch as a single tensor, we must pad shorter sentences with a special <PAD> token.
- **The Danger:** We do not want the model to learn to predict <PAD> tokens, nor do we want to penalize it for its guesses on padding.
- **The Solution (Padding Mask):** We calculate the loss for every position, but we multiply the loss at padded positions by 0.

$$\text{Total Loss} = \frac{\sum(\text{Loss}_t \times \text{IsNotPad}_t)}{\sum \text{IsNotPad}_t}$$

Training Step 4: Backpropagation & The Optimizer

Transformer training can be unstable early on due to high gradient variance and deep residual signal propagation.

- **Adam / Adam-W Optimizer:** Transformers almost universally rely on Adam or Adam-W, which maintain adaptive, per-parameter learning rates via moving averages of gradients and squared gradients.
- **Learning Rate Warmup:** A critical training trick. The learning rate is gradually increased (typically linearly) from a very small value over the first few thousand steps.
- **Why?** Early in training, attention patterns are essentially random. Large parameter updates during this phase can destabilize representation learning. Warmup limits update magnitude until attention mechanisms and internal representations begin to organize.

Remarks on the Training Pipeline

Modern Transformers are notoriously sensitive to initialization and optimization choices. The following "recipe" is used across BERT, GPT, and LLaMA families.

1. Optimization & Learning Rate
   - Optimizer (AdamW): Standard Adam can fail on Transformers. AdamW decouples weight decay from the adaptive gradient update, improving generalization.
   - LR Schedule (Warmup + Cosine Decay):
     - *Warmup:* Linearly increase the LR from 0 for the first 1–5% of steps. This prevents early divergence caused by the high variance of uninitialized LayerNorms and deep residual paths.
     - *Decay:* Smoothly anneal the LR down to $\sim$10% of its peak value using a cosine curve as the model converges.
2. Regularization & Stability
   - Gradient Clipping: Cap the global norm of gradients (usually to 1.0) to prevent exploding gradients during unexpected training loss spikes.
   - Weight Decay: Typically set around 0.01 to 0.1, but crucially, it is *never* applied to 1D parameters (biases and LayerNorm scales/shifts).

Inference Step 1: The Auto-Regressive Loop

Training is parallel, but generation is strictly sequential.

When deployed, we no longer have the target sequence.
Generation proceeds token-by-token.

Architecture Note:

- Decoder-only models (e.g., GPT): no Encoder.
- Encoder-Decoder models (e.g., T5): Encoder runs once, Decoder generates autoregressively.

1. Encode (if applicable): Run the Encoder once. Save H_{enc}.
2. Start: Feed $\langle$SOS$\rangle$ or prompt tokens into the Decoder.
3. Predict: Model outputs logits for the next token.
4. Select: Convert logits via Softmax and choose a token.
5. Append: Add token to the running sequence.
6. Loop: Repeat with causal masking until $\langle$EOS$\rangle$.

Key Constraint: Causal masking ensures each token attends only to previous tokens.

Inference Step 2: Decoding Strategies

How do we select from a vocabulary-sized distribution (30k–200k tokens)?

- Greedy Search: Take arg max at each step.
  - Fast but locally optimal decisions may hurt globally.
- Beam Search: Track top K candidate sequences.
  - Common in translation and structured tasks.
- Temperature Sampling:

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

  Higher $T \to$ flatter distribution $\to$ more randomness.
- Top-k Sampling: Sample only from top k tokens.
- Top-p (Nucleus) Sampling: Sample from smallest token set with cumulative probability $\geq p$.
  - Widely used in GPT-style systems.

Inference Step 3: The KV Cache (Optimization)

Naive decoding is inefficient.
Without caching, at timestep t, the model recomputes attention projections for all previous tokens.

KV Caching

- Store Keys and Values from previous steps in memory.
- At step t, compute K/V only for the newly generated token and append it to the cache.
- The new token attends to these cached representations.

Impact:

- Avoids redundant computation.
- Reduces inference latency dramatically.
- Essential for large-scale LLM deployment.

Note: Each new token still attends to prior tokens (linear in context length), but we avoid recomputing the full prefix each step.

The Evolution: Pretraining Large Transformers

The 2017 Transformer was built for translation.
Today, we pretrain massive variants to build Foundation Models.

Instead of task-specific training:

1. Pretrain: Learn general language representations using massive unlabeled corpora (self-supervised learning).
2. Fine-Tune / Align: Adapt for specific tasks or instructions.

Scaling Laws (Kaplan et al., 2020): Performance improves predictably with:

- Model size
- Dataset size
- Compute

Modern models use hundreds of billions of parameters and trillions of tokens.

Encoder-Only Architectures (e.g., BERT)

A model that reads the entire sequence at once (left-to-right and right-to-left) to build a deep, holistic understanding of the text. It acts as a powerful "reader" but can not "write" new text easily.

Architecture Design

- Drops the Decoder entirely.
- A deep stack of Encoder blocks.
- **Fully bidirectional:** Every word can attend to every other word simultaneously.

How it Learns (Pretraining)

- **The "Fill-in-the-Blank" Game** (Masked Language Modeling).
- We hide 15% of the words in a sentence (e.g., "The [MASK] chased the mouse").
- The model must guess the missing word using the surrounding context.

Best For: Tasks that require deep *understanding* rather than generation. Examples include categorizing emails (spam vs. not spam), sentiment analysis, and highlighting answers in a document.

Decoder-Only Architectures (e.g., GPT Family)

A model designed exclusively for generating text. It reads left-to-right and can not look ahead, making it the perfect engine for writing text one word at a time.

Architecture Design

- Drops the Encoder entirely.
- A stack of Decoder blocks.
- **Strictly Unidirectional:** It uses causal masking so a word can only "see" the words that came before it.

How it Learns (Pretraining)

- **The "Next Word Prediction" Game** (Causal Language Modeling).
- It is fed massive amounts of internet text and repeatedly asked: "Given this sentence so far, what is the exact next word?"

Best For: Open-ended generation, chatbots, writing code, and general-purpose reasoning.

Making GPTs Usable: Modern Enhancements

Raw "next-word predictors" are not very helpful as assistants. Modern models (like ChatGPT or Claude) use additional techniques to make them useful and safe:

- RLHF (Reinforcement Learning from Human Feedback):
  - Teaching the model to be polite, follow instructions, and avoid harmful answers by having humans rate its responses.
- RAG (Retrieval-Augmented Generation):
  - Giving the model a "search engine." Instead of relying purely on memorized facts (which causes hallucinations), the model retrieves real documents and summarizes them.
- Under-the-Hood Math Upgrades (RoPE, Grouped-Query):
  - Mathematical optimizations that allow models to process massive "context windows" (e.g., reading whole books at once) without running out of memory.

Encoder-Decoder Architectures (e.g., T5, BART)

The "Reader and Writer" setup. The Encoder reads the complex input, and the Decoder writes the new output. It is heavily optimized for transforming one type of text into another.

Architecture Design

- Uses both halves of the original 2017 Transformer.
- The **Encoder** processes the source text fully bidirectionally.
- The **Decoder** generates the target text one word at a time, checking back with the Encoder for context.

How it Learns (Pretraining)

- **The "Corrupted Text" Game**.
- Large chunks or phrases of text are deleted or scrambled.
- The Encoder reads the broken text, and the Decoder must generate the fixed, original text.

Best For: Sequence-to-sequence transformation tasks. Examples include translating English to French, or summarizing a 10-page article into a 3-bullet-point summary.

Beyond the Vanilla Transformer

Modern innovations include:

- Mixture-of-Experts (MoE)
  - Activate subset of parameters per token.
  - Enables trillion-parameter scale efficiently.
- Long-Context Techniques
  - FlashAttention (hardware-aware memory optimization)
  - Linear attention variants
  - 100k+ token context windows
- Speculative Decoding
  - Small draft model proposes tokens.
  - Large model verifies them.
  - Accelerates inference.
- Multimodal Models
  - Unified text, vision, and audio architectures.